CIS 4910

Computer Science & Engineering Project: Capstone

Planning, Risk, Testing and Standards in Practice

From Ideas to project plans and test plans to risk management and engineering standards; how it all comes together in a single specification.

WEEK 4 • FALL 2026

Professor Dr. Ken Christensen
scoth@usf.edu • Office Hours by appointment (Microsoft Teams)

Bellini College of Artificial Intelligence, Cybersecurity and Computing

TODAY'S SESSION

Agenda

1. Video: Why → How → What
2. Project Management
3. Project Plans & Schedules
4. Risks
5. Project Constraints
6. Engineering Standards
7. Test Plans
8. High Level Architecture
9. Putting It All Together: One Specification

CLASS DISCUSSION

Why, How, What

Watch Before We Discuss

Simon Sinek on the order of creating a useful product

Why, How, What

What a business leader needs from its engineers

Questions I Will Ask each team Next Week

1. Why is your product needed?
2. How does your product solve your customer's problem?
3. What does your product do?

FROM SPONSOR TO SOLUTION

Why → How → What

Use the conversation to keep the team from jumping straight to implementation.

WHY

What business problem exists?

Who is affected?

What value does success create?

HOW

What capabilities could solve the problem?

What constraints matter?

What must be true?

WHAT

What will the customer actually receive?

How will we know it works?

Do not let “What can we build?” replace “What problem are we solving?”

CLASS DISCUSSION

What Do You Need From Me Right Now?

Four weeks in, with the SRS deadline approaching in Week 5, what do you need?

? Blockers

What is stopped, broken, or missing that I can unblock?

? Clarity

What requirement or expectation is still unclear?

? Support

Where do you need a decision, or guidance?

SECTION 2

Project Management

Why structured management matters once a team, not just an individual, owns the outcome.

What Project Management Means

- Project management is the discipline of planning, organizing, and overseeing the work needed to deliver your capstone project on time and to spec.
- In industry this is often a dedicated role. In your capstone team, it is a shared responsibility that rotates or is owned by one team member.
- Good project management does not slow a team down. It prevents the last-minute scramble that happens when nobody tracked what was actually done.
- The plan, the schedule, the test plan, the risk list, exists to make the project's status visible to everyone on the team.
- Generally, a "Sprint" is 2 weeks, for this class the sprint = 1 week.

Core Roles & Responsibilities

L

Team Lead / Coordinator

Keeps the schedule current, runs check-ins, and is the point of contact with the instructor.

D

Technical Owner(s)

Responsible for architecture decisions and code quality on their assigned components.

Q

Quality / Test Owner

Maintains the test plan and verifies acceptance criteria before work is marked done.

What Is a Story Point?

A singular, relative number used to estimate the combination of volume, complexity, Risk, and knowledge of User Stories.

WHAT POINTS MEASURE

E **Effort or Volume**
How much work is needed

C **Complexity**
How difficult is the implementation

U **Risk & Uncertainty**
What unknowns or roadblocks exist

K **Knowledge & Experience**
How well is the technology understood

HOW TEAMS USE POINTS

1 2 3 5 8 13

Fibonacci scale — gaps force honest estimation

- Plan in points, schedule in sprints. Points stay stable; dates shift.
- Never assign points alone. Estimation is a team conversation.
- AI can suggest point values; treat them as an anchor, not a verdict.
- An 8-point story should be split. Cannot estimate it? You don't understand it yet.

The Dreaded Story Point

Story points estimate relative effort and uncertainty, not hours.

1 POINT

Small, familiar, low uncertainty

Example: simple field validation

3 POINTS

Moderate work or some unknowns

Example: new API endpoint

5 POINTS

Several moving parts or learning

Example: external service integration

8+ POINTS

Too large or too uncertain

Break it into smaller stories

A story point is a planning signal, not a promise of elapsed time.

If you must think in terms of time: 1sp is roughly 4 to 12 hours.

AI as a Project Management Assistant

AI

Where AI Genuinely Helps

- **Status synthesis:** feed AI your team's raw stand-up notes and ask it to draft a clean status summary for the instructor.
- **Meeting notes:** AI can turn a messy meeting transcript into action items with owners and due dates.
- **What AI should NOTE do:** make the actual prioritization decisions. That judgment call stays with the team.

SECTION 3

Project Plans & Schedules

Turning a list of tasks into a timeline the whole team can actually follow.

What a Project Plan Contains

S

Scope

What is in the project and, just as important, what is explicitly out of scope for this semester.

M

Milestones

The major checkpoints (prototype, alpha, beta, final demo) with target dates. These are set in the course syllabus. (Week 5, 8, 12, 15)

D

Deliverables

The concrete artifacts due at each milestone: code, documentation, a working demo.

Building a Schedule

- Start from milestones, then break each one into tasks small enough to estimate in days, not weeks. The major milestones are already provided by when assignments are due
  - ✓ Week 3: Short requirements presentation. Should have a good idea and layout of the PP and development environment / CICD pipeline, etc. First team status report complete.
  - ✓ Week 5: SRS Version 1 is due: Should also have a good working set of web pages or app layout
  - ✓ Week 8: DDD Version 1 is due: App should be operational with at least 50% of the requirements met
  - ✓ Week 12: SRS/DDD should be all but complete. App should be 75-80% complete : Test plans added here.
  - ✓ Week 15: Production demo to class and sponsors; final versions of the SRS/Test Plan & DDD
- Identify dependencies: which tasks must finish before another can start. This is what creates your critical path and helps identify risks.
- The critical path is the longest chain of dependent tasks. It determines your earliest possible finish date, and any delay on it delays the whole project.
- Build in buffer time. A schedule with zero slack breaks the first time anything goes wrong, and something always goes wrong.

The Schedule and Milestones

A schedule allocates resources across time. Every activity ends with a measurable milestone a verifiable deliverable, not a description of activity.

What Makes a Good Milestone?

Bad: Finish testing

Good: 1,000 test cases run with zero severity-1 defects open.

Bad: Complete the design

Good: Architectural Design Document signed off by all team members.

Bad: Work on the SRS

Good: SRS v1.0 submitted to Canvas on Wednesday 11:59 PM.

AI-Assisted Estimation & Dependency Checking

- Estimation: describe a task to an AI assistant and ask for a rough time estimate based on similar work. Treat it as a starting point, not a guarantee.
- Dependency checking: paste your task list and ask AI to flag tasks that look like they depend on something not yet listed.
- The team still owns the final schedule. AI is a second pair of eyes, not the planner of record.

Disclose it

If AI helped build your schedule, say so in your status report, the same way you would cite any other tool or paper. (APA is preferred, IEEE allowed)

How Schedules Are Built and How They Slip

How Schedules Are Built

E **Past Experience**
Works best in mature disciplines with similar prior projects.

C **Complexity Analysis**
Estimate effort using a complexity metric, then calibrate against team velocity.

P **Political or Market Dictate**
The deadline is set externally, and the team works backwards Kennedy's moon commitment is the classic example.

How Schedules Slip

1. **One Day at a Time**
Schedules rarely collapse overnight they erode one small excuse per day. **No Buffer Built In**

2. A schedule with zero slack breaks the first time something goes wrong. Something always goes wrong. **No Incremental Check**

3. Without iterative delivery there is no way to know you are off track until it is too late.

What to Do When a Schedule Slips

The correct response is never to miss the deadline it is to negotiate scope and deliver on time with less functionality.

1	**Pull Scope, Not Time**	Remove the lowest-priority features. Deliver on schedule, then negotiate the rest.
2	**Communicate Early**	Tell your sponsor the moment you know you are behind. A surprise miss is far worse.
3	**Revise the Plan**	Update the schedule to reflect reality. An incorrect plan is not a plan; it is wishful thinking.
4	**Learn from the Slip**	Run a retrospective after delivery. What caused the slip? What changes next sprint?

In this class: delivering a perfect project after the final demo date means you have already failed.

SECTION 4

Risks

What could go wrong, how likely it is, and what you will do about it before it happens.

What Is Risk Management

- A risk is anything that could negatively affect your project's scope, schedule, or quality, but has not happened yet.
- Risk management is the practice of identifying these threats early, judging how serious they are, and planning a response in advance.
- The goal is not to eliminate all risk. Some risk is unavoidable. The goal is to never be surprised by something you could have anticipated.
- A risk you wrote down and planned for is a manageable problem. A risk you never considered is a crisis.

Identifying & Categorizing Risks

T

Technical

A chosen technology, library, or API turns out not to work the way you expected.

S

Schedule

A dependency runs late, or a team member underestimates how long a task will take.

R

Resource

A team member becomes unavailable, or you lose access to required hardware or accounts.

C

Scope

Requirements grow beyond what was originally agreed, often called scope creep.

The Risk Matrix: Likelihood x Impact

- Rate each risk on two dimensions: how likely is it to happen, and how bad would it be if it did.
- A risk that is both likely and high-impact needs a plan now. A risk that is unlikely and low-impact may just need to be logged and watched.



Mitigation Strategies

A

Avoid

Change your approach to eliminate the risk entirely, such as choosing a more stable library.

R

Reduce

Take steps that lower the likelihood or impact, such as a proof-of-concept spike before full implementation.

T

Transfer

Shift the risk elsewhere, such as relying on a managed cloud service instead of self-hosting.

X

Accept

Acknowledge the risk and plan a fallback, used when the cost of avoiding it outweighs the risk itself.

AI-Assisted Risk Brainstorming

- Describe your project's architecture and plan to an AI assistant and ask it to brainstorm risks you may not have considered.
- AI is good at pattern-matching against common failure modes: third-party API changes, single points of failure, untested assumptions.
- The team still rates likelihood and impact and decides on a response. AI expands the list, it does not own the judgment call.

> **Cast a wide net, then filter**
>
> Let AI over-generate candidate risks. Your team's job is to filter for what is actually plausible.

Risk Management for Your Project

In Class, Identify roughly five risks in priority order. A risk you planned for is a manageable problem; one you never considered is a crisis.

1	**Identify**	Name the risk. What could go wrong — technical, schedule, resource, or scope?
2	**Assess**	Rate it on likelihood and impact. High-likelihood, high-impact risks need a plan now.
3	**Mitigate**	Avoid, Reduce, Transfer, or Accept — choose a response proportional to the risk.
4	**Track**	Keep the risk list alive and review it every sprint. A closed risk that resurfaces is the most dangerous kind.

Categories: Technical • Schedule • Resource • Scope (Creep)

Risk Examples from Previous Semesters

These are real events. Each one could have been identified and mitigated in advance.

Missing Device Driver

Technical Risk — High Likelihood Once Identified

A team built hardware assuming a PC device driver would exist. It did not. The final project could not meet requirements.

Lesson: Verify driver availability before committing to a hardware interface or specify a fallback.

Crashed Drone

Resource Risk — Low Likelihood, Catastrophic Impact

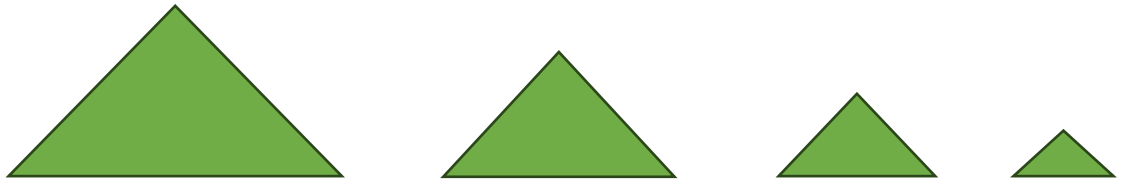
A team crashed their drone before the key demo. Because they had recorded video of the drone working earlier, they were able to present that footage as proof of functionality.

Lesson: Recording a working prototype early is a risk mitigation strategy for critical demo assets.

SECTION 5

Project Constraints

The boundaries every decision on your project has to work within.



The Constraint Triangle: Given Scope: Time, Cost, Quality

- Every project is bounded by three interacting constraints based on the scope
- **time** (how long you have), **cost** (the resources available), **quality** (How many bugs go to prod)
- Change one and the others move. Add scope without adding time or cost, and quality usually pays the price.
- For your capstone, **time is fixed** by the semester calendar, and cost is largely fixed by the team you have. **That leaves Scope Vs Quality as your main levers.**
- **the lever to pull is *SCOPE*, never quality, rarely time or cost.**

PROJECT CONSTRAINTS

Real-World Constraints on a Capstone Team

S

Team Skill

Your plan has to match what the team can actually build, not what would be ideal with unlimited expertise.

T

Tools & Access

Available licenses, hardware, and APIs. A great design that needs a tool you cannot access is not viable.

A

Academic Calendar

Fixed milestones, demo dates, and finals week do not move. Plan around them, not against them.

AI-Assisted Constraint Analysis

- Describe your team’s skills, tools, and calendar to an AI assistant and ask it to flag constraints you may be underestimating, such as a licensing cost or an access restriction.
- AI is useful for surfacing trade-offs: ask what typically breaks when a team cuts scope vs. quality vs. time on a similar project.
- The team still decides which lever to pull. AI can list options; it cannot tell you what your sponsor will actually accept.

Disclose it

If AI helped you reason through a constraint or trade-off, say so in your status report or SRS, the same way you would cite any other tool or source. (APA is preferred, IEEE allowed)

SECTION 6

Engineering Standards

The shared rules that make your work safe, accessible, and compatible with everyone else’s.

ENGINEERING STANDARDS

Standards You Will Actually Encounter

L

ISO/IEC/IEEE 12207

The software life cycle standard. Defines the processes every team runs through, from requirements to retirement, in both Waterfall and Agile contexts.

T

ISO/IEC/IEEE 29119

The software testing standard. Defines vocabulary, test processes, and documentation. Directly underlies the test plan structure from Section 3.

A

WCAG 2.1, Level AA

The DOJ's ADA Title II baseline for public entities (deadlines phased in through 2027-2028). Covers contrast, keyboard navigation, screen readers.

S

OWASP Top 10:2025

The current ranked list of critical web security risks. Top two: Broken Access Control and Security Misconfiguration.

These are just a few, there are Many others!

Standards Categories Relevant to Your Capstone

C

Coding & Style

Language style guides and linting rules that keep a shared codebase consistent and maintainable.

S

Security

Standards like OWASP for web application security, governing how you handle authentication and data.

A

Accessibility

WCAG guidelines ensure your interface is usable by people with disabilities.

I

Industry-Specific

Domain standards for your field, such as HL7 for healthcare or IEEE for networking.

AI-Assisted Standards Research

- Ask an AI assistant to summarize a standard such as WCAG, OWASP, or ISO/IEC/IEEE in plain language, or to check a snippet of your code against a specific rule.
- AI is useful for a first pass: flagging likely accessibility issues in your markup or common OWASP-listed mistakes in your authentication flow.
- Never cite a standard's version, date, or exact threshold from AI memory alone. AI can misstate these. Verify against the primary source before it goes in your SRS.

Verify before you cite it

An AI-summarized standard is a starting point, not a citation. Confirm the exact standard, version, and requirement against its official source before it appears in your SRS.

SECTION 7

Test Plans

Deciding, before you write code, what "it works" actually means.

NOTE: When organizations get to “exacting” in this area, what emerges is the concept of “Test Driven Development or TDD”.

What Is a Test Plan, and Why It Exists

- A test plan is a written description of what will be tested, how, and what counts as passing. There must be a test that verifies each requirement / specification.
- It exists so that "done" has a verifiable definition instead of being a feeling someone has on demo day.
- Writing it before you build forces you to think through edge cases and failure modes early, when they are cheap to fix.
- A test plan is also evidence: it is what you show an evaluator / auditor to prove the system was actually verified, not just demoed once and hoped for.
- For this class, your test plan is a section of the SRS document; it is not stand alone. Some organizations & classes treat the TP as a separate document; I do not.

Test Plan Components

S

Scope

What features or components are covered by this test plan, and what is explicitly excluded.

C

Test Cases

Specific inputs, steps, and expected outputs for each scenario being verified.

A

Acceptance Criteria

The pass/fail threshold. What result counts as the feature working correctly.

E

Environment

The hardware, software versions, and data needed to run the tests consistently.

Levels of Testing

1. **Unit Testing**
   Verifies a single function or component in isolation. Fast, narrow, run constantly.

2. **Integration Testing**
   Verifies that multiple components work correctly together once connected.

3. **System Testing**
   Verifies the whole application end to end, as a complete product.

4. **Acceptance Testing**
   Verifies the system against the user story’s acceptance criteria.

AI-Assisted Test Case Generation

- Give an AI assistant a function signature or a user story and ask it to draft test cases, including edge cases you might not think of.
- AI is especially useful for boundary conditions: empty inputs, maximum values, malformed data, concurrent access.
- Always review generated test cases yourself. AI can generate a test that passes trivially without actually verifying the right behavior.

> **A generated test is a draft**
>
> Treat AI-written test cases as a first draft for your review, not a finished test plan.

Black Box vs. White Box Testing

It is impossible to uncover every failure through testing. The goal is to design test cases that cover as much of the system as possible.

Black Box Testing

Tests ***all system functionality*** as seen by the end user no knowledge of internal code.

Covers every feature, input, output, and error path the user can reach.

Your SRS test cases are primarily black box at this stage.

White Box Testing

Tests the ***internal logic of the code*** itself the tester has full source code access.

Covers every branch, loop, and line of written code.

AI-generated code especially needs white box review.

What Is a Test Case?

A test case should be executed by someone who knows little about the project.

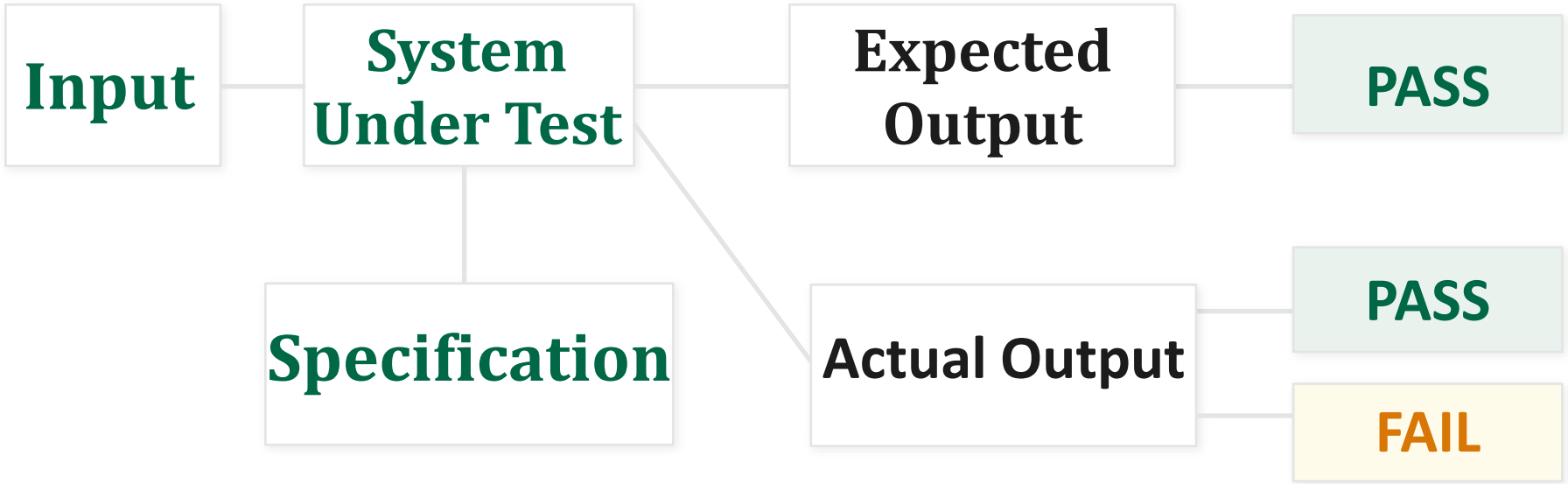
1 Coverage	Which requirement and specification item does this test verify? Reference the ID number.
2 System State	The exact starting condition of the system before the test begins not a guess.
3 Input	The exact input provided: keystrokes, API calls, files, or sensor values. No ambiguity.
4 Expected Output	The exact result the system must produce. A specific value or state not just a feeling.

Pass/Fail: Actual output equals expected output exactly — any deviation is a fail and must be logged.

What Goes in a Test Plan?

S **Test Strategy**	Describe your overall approach which testing types you will use, and in what order.
D **Design of Experiments**	List all inputs that will be tested: typical cases, error cases, and edge cases.
C **Input Combinations**	Define which combinations of inputs will be exercised shown in a table or prose.
L **Scope Declaration**	Be explicit about what you will not test you cannot test every possible input.
E **Environment**	The hardware, software versions, OS, and data state required to run tests consistently.

Anatomy of a Test Case



The Pass/Fail Rule

If actual output equals expected output exactly, the test passes. Any deviation is a failure even a minor formatting difference. A failed test does not mean the project has failed. It means the spec, the code, or both must be revisited before the test is re-run.

SECTION 8

High Level Architecture (HLA)

A 1-page diagram of your application and its components.

NOTE: The HLA lives in the DDD for week #8 deliverable. I like to see an attempt in the SRS, not perfect, but something that says you understand the movement of logic and data.

Lots of EA’s will insist on a HLA at a very early stage!

SRS SECTION

High Level Architecture (HLA)

The HLA (usually 1 page) explains the overall structure of a system, the main components, how they interact, and how data flows. **It must be in your SRS for the Week 5 submission; then removed. Moved to DDD thereafter!**

B Block Diagram

Show each major component as a labeled box with lines indicating data flow or control.

I Interface Definitions

For each connection in the diagram, define what crosses it format, protocol, and direction. Remember “High Level”, not detailed!

S System Boundaries

Clearly mark what is inside your system and what is external: APIs, databases, third-party services.

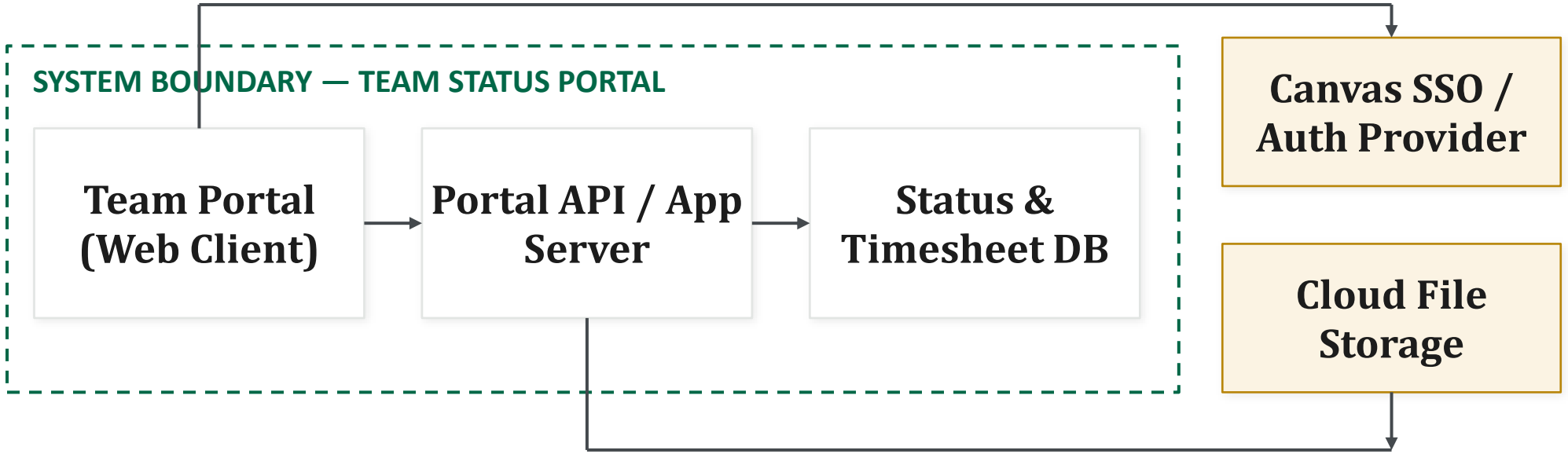
C Component Descriptions

Describe each block's responsibility: what it does, consumes, and produces.

SRS SECTION

A Worked HLA Example

The Week #5 status-report feature from slide 45, drawn as a one-page block diagram.



Interface Definitions — What Crosses Each Connection

- Client ↔ API: HTTPS/REST + JSON request/response, both directions
- API → Database: SQL queries, internal to the system boundary only
- API → Cloud Storage: HTTPS multipart upload of PDF, Word, and Excel files
- Client ↔ Auth Provider: OAuth2/SSO handshake, external, both directions

AI-Assisted HLA Drafting

- Describe your components and data flow in plain language and ask an AI assistant to draft a first-pass block diagram or clean up your labels and terminology.
- AI is good at suggesting a standard shape for your architecture style, such as client-server, pipeline, or event-driven, that you may not have considered.
- AI cannot see your actual codebase. A generated diagram can look clean and still be wrong. The team verifies every box and every connection against what you are actually building.

Disclose it

If AI helped draft or refine your HLA diagram, say so in your SRS, the same way you would cite any other tool or source. (APA is preferred, IEEE allowed)

SECTION 9

Putting It All Together

Every section from today, applied to one real specification, written as user stories.

The Finished Specifications (WEEK #5: Goal)

1.0: Requirement: **As a** team member, **I want to** upload my weekly status report through the team portal, **so that** the instructor can review progress without needing email.

- **Constraints:** Must ship before Wednesday due dates.
- **Assumptions:** The due date should not change.
- **Acceptance**: Report appears in the review queue within one minute of submission and before due date.
- **Standards applied:** WCAG accessibility, team coding style guide.
- **Test plan:** unit, integration, and acceptance tests defined per the criterion above.
- **Top risk:** storage API rate limits, mitigated with retry logic.

a) **Specification: As a** team member, **I want** to upload my weekly status report as a PDF or MS Word Document and the Timesheet as an Excel using both templates provided in Canvas, **so that** the instructor can quickly and easily review the report without any future communication.

Week 3 slides also have a detailed example of both requirements and specifications expected.

Tasks for the coming Week #5

1. **Ensure all Requirements and specifications are tracked; Both written as Stories!**

2. **Use the *templates* in Canvas for the SRS Word Document, PPT Presentation and TIMESHEET! Remember, week 5 Submission includes the TEAM Video from week 3.**

3. **Use the discussion board for major assignment questions and issues.**

4. **Practice your SRS presentation and ensure it is polished!**

5. **All Team members must be present and take a role in the presentation.**